Esercitazione 22 aprile

Overload di operatori

1

Un po' di teoria [*Prova pratica del 23 luglio 2018, Testo 1, Esercizio 2*]

Identificare e correggere gli errori nella seguente dichiarazione di classe C++:

```
1. class Prova {
2. public:
3.     Prova();
4.     Prova(double val);
5.     Prova(const Prova& p);
6.     double getVal();
7.     void setVal();
8.     Prova operator+(Prova p);
9.     operator=(const Prova& p);
10.    istream& operator>>(istream& is, Prova& p);
11.private:
12.    double _val;  };
```

2

Un po' di teoria [*Prova pratica del 23 luglio 2018, Testo 1, Esercizio 2*]

Identificare e correggere gli errori nella seguente dichiarazione di classe C++:

```
1. class Prova {
2. public:
3.     Prova();
4.     Prova(double val);
5.     Prova(const Prova& p);
6.     double getVal() const;
7.     void setVal(double val);
8.     Prova operator+(Prova p);
9.     const Prova& operator=(const Prova& p);
10.    friend istream& operator>>(istream& is, Prova& p);
11.private:
12.    double _val;  };
```

3

[Esercizio C065] *Prova pratica del..*

Si scriva, in linguaggio C++, l'interfaccia della classe **Campione**, contenente i dati di temperatura e pressione di un liquido. In particolare, la classe sarà dotata dei tre attributi: un *codice* identificativo del campione (un numero intero positivo), il valore della *temperatura* espresso in gradi Celsius (un numero reale) e il valore della *pressione* espresso in bar (un numero reale positivo).
La classe dovrà inoltre prevedere i seguenti metodi:

- il costruttore di default e costruttore di copia;
- un costruttore con parametri che riceva i valori del codice identificativo, della temperatura e della pressione;
- il distruttore;
- i metodi selettori (uno per ciascun attributo);
- i metodi modificatori (uno per ciascun attributo);
- l'operatore == per verificare se due campioni sono identici, cioè se hanno lo stesso codice, la temperatura e la pressione;
- l'operatore = per assegnare a un oggetto i valori degli attributi di un altro oggetto della stessa classe;
- l'operatore != per verificare se due campioni sono diversi;
- l'operatore > per verificare se un campione ha pressione maggiore di un altro;
- l'operatore < per verificare se un campione ha pressione minore di un altro;
- l'operatore binario - per calcolare la differenza tra le temperature di due campioni;
- l'operatore unario - per ottenere un nuovo campione con temperatura opposta rispetto a quella del campione dato, lasciando invariati gli altri attributi;
- l'operatore di estrazione >> e l'operatore di inserimento << per leggere e stampare i dati di un campione;
- un metodo che converta la temperatura in kelvin ed un altro metodo che converta la pressione da bar a pascal.
- Inoltre, si definisca una **funzione amica** che calcoli la temperatura media di due campioni.

4

[Esercizio C062] Punto3D

Implementare la classe *punto3d* con i seguenti metodi:

- Il costruttore di default che inizializza a zero tutti i attributi,
- Un costruttore con parametri che riceve in ingresso (ovvero come parametri) tre numeri reali che rappresentano le coordinate *x, y e z* di un punto 3D e assegna i valori dei parametri attuali ai corrispondenti attributi della classe
- Il costruttore di copia
- Il distruttore
- I selettori (un metodo per ciascun attributo)
- I modificatori (un metodo per ciascun attributo)

5

[Esercizio C062] Punto3D

Implementate i seguenti operatori:

- L'operatore unario di cambio di segno (-), per calcolare l'opposto di un punto 3D.
- Gli operatori unari di incremento e decremento unitario prefisso e postfisso. (++, --)
- Gli operatori aritmetici di somma, sottrazione e moltiplicazione per scalare (+,-,*)
- L'operatore di assegnamento (=)
- L'operatore relazionale di uguaglianza ==, per il quale due punti 3D sono uguali se hanno le stesse coordinate.
- Gli operatori di inserimento ed estrazione (>>,<<)

- Un funzione esterna amica per il calcolo della distanza tra due punti.

Si scriva quindi un programma C++ per verificare il corretto funzionamento della classe sviluppata.
Il programma dichiarerà due oggetti di classe *punto3D*, chiederà all'utente di immettere da tastiera i valori delle coordinate dei due punti utilizzando l'operatore di inserimento, stamperà a video le coordinate immesse utilizzando l'operatore di estrazione e calcolerà e stamperà a video la somma, la differenza, e la distanza tra i due punti.

6

[Esercizio C063] Utilizzo di classe Punto3D

Assumiamo che il Punto3D corrisponde alla posizione della mano misurata con il sistema di cattura di movimento.
Il sistema è in grado di rilevare 15 volte al secondo la posizione della mano. Implementare un programma in C++ che:

- calcola la velocita della mano sulle finestre di 15 misurazioni (1 secondo).
- legga da un file di testo un insieme di punti 3D e calcoli la velocità media su finestre di 15 punti utilizzando la tecnica di finestra scorrevole.

Il programma deve:

- aprire il file di input contenente i punti 3D.

- allocare dinamicamente di un array di punti 3D.
- leggere dei punti dal file e inserire nell'array dinamico.

- calcolare la velocità media su finestre di 15 punti utilizzando la tecnica di finestra scorrevole.
- stampare la velocità media per ogni finestra di 15 punti sulla console.

7

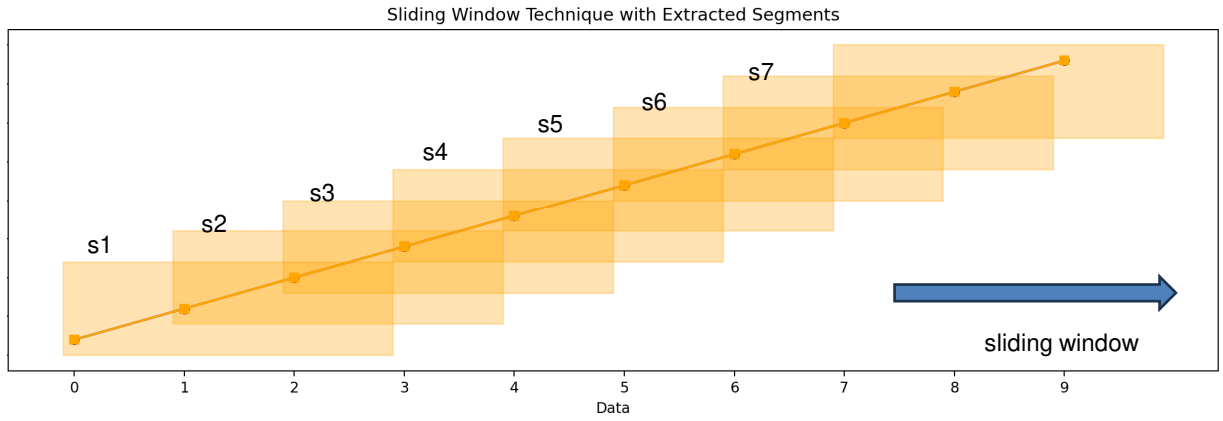
[Esercizio C063] Utilizzo di classe Punto3D

Esempio del file punti3D.txt:

0 0 0
0.000109739 0.000109739 0.000109739
0.000548697 0.000548697 0.000548697
0.00153635 0.00153635 0.00153635
0.00329218 0.00329218 0.00329218
0.00603567 0.00603567 0.00603567
0.00998628 0.00998628 0.00998628
0.0153635 0.0153635 0.0153635
0.0223868 0.0223868 0.0223868
0.0312757 0.0312757 0.0312757
0.0422497 0.0422497 0.0422497
0.0555281 0.0555281 0.0555281

8

- Segmenti a durata fissa senza considerare la semantica:
- la “finestra scorrevole” consiste nello spostare una finestra di dimensione fissa sui dati per analizzare o elaborare in modo sequenziale sottosezioni dei dati; è spesso utilizzata quando è disponibile solo una quantità limitata di dati.



9

[Esercizio C064] Poligono

Progettare e sviluppare in linguaggio C++ la classe *poligono2d* per rappresentare poligoni i cui vertici sono punti bidimensionali. Il poligono è rappresentato come un array di punti bidimensionali, ovvero un array di oggetti della classe *punto2d*

La classe avrà pertanto i seguenti attributi:
- il puntatore *_p* ad un array di oggetti di classe *punto2d*
- il numero *_n* di punti contenuti nell’array (un numero intero).

Si implementino, inoltre, i seguenti metodi (1/4):
- Il costruttore di default che inizializzi *_p* a NULL e *_n* a zero.

- Un costruttore con parametri che riceva in ingresso (ovvero come parametri) un numero intero *dim*. Il costruttore effettuerà l’allocazione dinamica di un array di *dim* oggetti di classe *punto2d*, ne assegnerà il puntatore a *_p* e assegnerà *dim* a *_n*. Nel caso in cui l’allocazione dinamica della memoria non vada a buon fine o il valore di *dim* non sia valido, il costruttore inizializzerà *_p* a NULL e *_n* a zero.

10

[Esercizio C064] Poligono

Si implementino, inoltre, i seguenti metodi:

- Un costruttore con parametri che riceva in ingresso (ovvero come parametri) un array *pti* di oggetti di classe *punto2d* e la sua dimensione *dim* (un numero intero). Il costruttore opererà come segue:
  - nel caso in cui *dim* assuma un valore valido, effettuerà l'allocazione dinamica di un array di *dim* oggetti di classe *punto2d* e ne assegnerà il puntatore a *_p*;
  - nel caso in cui l'allocazione dinamica della memoria vada a buon fine, assegnerà a ciascun elemento dell'array puntato da *_p*, il valore del corrispondente elemento dell'array *pti* e assegnerà *dim* a *_n*;
  - nel caso in cui l'allocazione dinamica della memoria non vada a buon fine o il valore di *dim* non sia valido, il costruttore inizializzerà *_p* a NULL e *_n* a zero.
- Il costruttore di copia. Nel caso in cui l'allocazione dinamica della memoria non vada a buon fine, il costruttore inizializzerà *_p* a NULL e *_n* a zero.
- Il distruttore.
- I *selettori* (un metodo per ciascun attributo).

11

[Esercizio C064] Overload di operatori

- Il modificatore per l'array puntato da *_p* che riceva in ingresso (ovvero come parametri) un array *pti* contenente i nuovi valori e la sua dimensione *dim* e operi come segue:
  - nel caso in cui il valore di *dim* sia valido, dichiari un puntatore *p* a un array di oggetti di classe *punto2D*, effettui l'allocazione dinamica di un array di *dim* oggetti di classe *punto2D* e ne assegni il puntatore a *p*,
  - nel caso in cui l'allocazione dinamica della memoria vada a buon fine, assegni a ciascun elemento dell'array puntato da *p*, il valore del corrispondente elemento dell'array *a*, deallochi (se esiste) l'array puntato da *_p*, assegni il valore di *p* al puntatore *_p* e assegni *dim* a *_n*;
  - nel caso in cui l'allocazione dinamica della memoria non vada a buon fine o il valore di *dim* non sia valido, il modificatore non opera alcuna modifica ai dati.
- L'operatore di moltiplicazione per scalare,
- L'operatore di assegnamento. Nel caso in cui l'allocazione dinamica della memoria non vada a buon fine, l'operatore assegnerà NULL a *_p* e zero a *_n*,
- L'operatore di selezione con indice,
- L'operatore relazionale di uguaglianza, per il quale due poligoni sono uguali se sono uguali le coordinate di tutti i punti che ne costituiscono i vertici,

12

[Esercizio C064] Overload di operatori

Si implementino, inoltre, i seguenti metodi:

- Gli operatori di inserimento ed estrazione.
- Un metodo per calcolare il perimetro del poligono come somma delle distanze tra coppie di vertici consecutivi.

Si scriva infine un programma C++ per verificare il corretto funzionamento della classe sviluppata.

Il programma:

- chiederà all'utente di inserire da tastiera il numero di vertici desiderato per il poligono,
- dichiarerà un oggetto di classe *poligono2d* passando al costruttore il numero di vertici desiderato,
- chiederà quindi all'utente di inserire da tastiera le coordinate dei vertici del poligono utilizzando l'operatore di estrazione,
- calcolerà infine il perimetro del poligono e ne stamperà a video il valore.

13